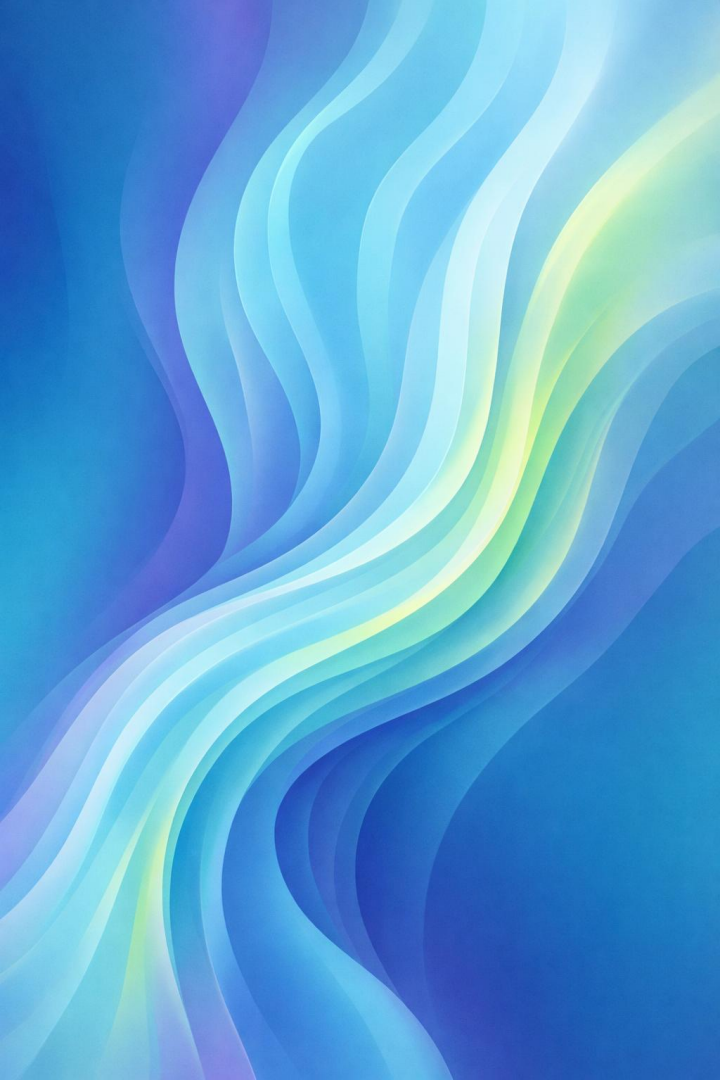


OpenFOAM

Thermal Simulation of a CPU Cooler: Modeling Heat Sources and Fans (Part 1)

Nicola Andreini, PhD

OVERVIEW

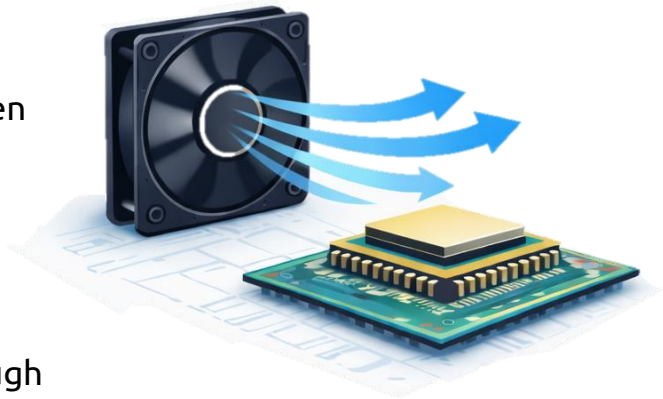
OpenFOAM V2406

A practical LinkedIn series to simulate real-world engineering scenarios with OpenFOAM.

Focused on CFD for **thermal management**, **multiphase flow**, phase changephenomena , and **common tasks**.

Goal :

Simulate airflow from a fan and predict chip temperature under a given heat load



What you'll see:

- How to model fan performance using characteristic curves in boundary conditions
- How to define time-dependent heat sources on solid bodies through *fvOptions* dictionaries

If you find this content valuable : **share the post** and **leave a comment** with real-world cases, **you'd like to see covered**

IMPLEMENTATION

constant / solid / fvOptions

OpenFOAM V2406

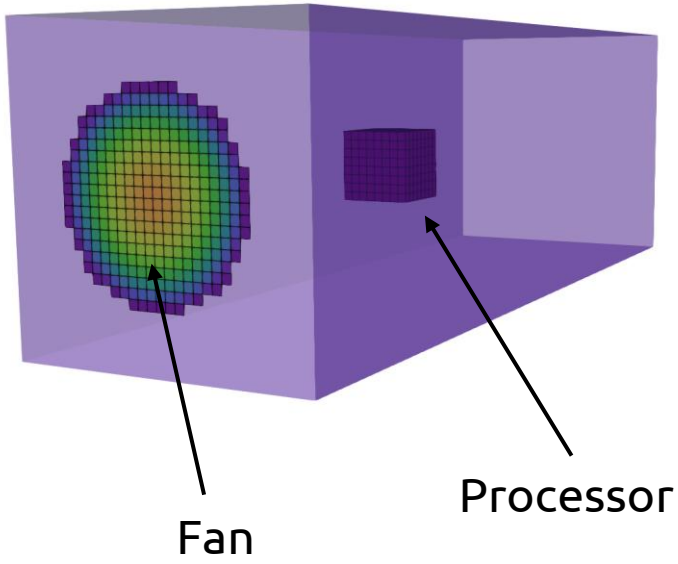
- **chtMultiRegionFoam**, a conjugate heat transfer solver, was used for this case
- The processor chip generates heat uniformly throughout its volume modeled as a volumetric source term in the energy equation $Sh_{\max} = \frac{P}{V}$
- For **heSolidThermo** in chtMultiRegionFoam, we use **scalarCodedSource**
- **codedAddSupRho** defines a source term for the enthalpy equation, activated by the *fvOptions(rho, h)* call in *solveSolid.H*.

1

```
heaterSource
{
    type            scalarCodedSource;
    selectionMode   all;
    fields          (h);
    name            timeVaryingHeatSource;

    codeInclude
    #{
    #};

    codeCorrect
    #{
    #};

    codeConstrain
    #{
    #};
```

2

```
codeAddSupRho
#{
    // Access cell volumes
    const scalarField& V = mesh_.V();

    // Access equation source term
    scalarField& heSource = eqn.source();

    // Access current simulation time
    const scalar t = mesh_.time().value();

    // Input parameters
    const scalar totalPower = 10.0;    // [W] total chip power
    const scalar rampTime = 2.0;       // [s] ramp duration

    // Compute total volume of the source region
    scalar totalVolume = 0.0;
    forAll(V, i)
    {
        totalVolume += V[i];
    }

    // Compute power density from total power
    const scalar maxPowerDensity = totalPower / totalVolume;  // [W/m³]

    // Compute current power density based on time ramp
    scalar currentPowerDensity = 0.0;
```



fvOptions provides a flexible framework to add source terms to transport equations without modifying the solver.

IMPLEMENTATION

constant / solid / fvOptions

OpenFOAM V2406

```
        if (t < rampTime)
        {
            // Linear ramp
            currentPowerDensity = maxPowerDensity * t / rampTime;
        }
        else
        {
            // Constant value after ramp
            currentPowerDensity = maxPowerDensity;
        }

        // Apply source to all cells
        forAll(heSource, i)
        {
            heSource[i] -= currentPowerDensity * V[i];
        }

        // Debug output
        Info << "Time: " << t << " s | Power density: "
             << currentPowerDensity << " W/m³ | Total power: "
             << currentPowerDensity * totalVolume << " W" << endl;
    #};
}
```

3

forAll(heSource, i): OpenFOAM macro that iterates over all field elements, equivalent to *for (label i = 0; i < heSource.size(); i++)*

currentPowerDensity * V[i]: converts power density [W/m³] to absolute power [W] for that cell

-= sign: the minus sign is required because in OpenFOAM the source term in the equation Ax = b is moved to the left-hand side

What will be covered in Part 2 ?

Part 2 will focus on implementing the fan performance curves and creating the patch where the fan boundary condition will be applied.

FOLLOW FOR MORE